
ForL0 State Backend

编译构建指导书

2026 年 8 月

目 录

1	文档概述	1
1.1	目标平台	1
1.2	构建产物	1
2	基础环境准备	1
2.1	系统构建工具	1
2.2	JDK 与 Maven	2
2.3	Apache Flink 运行环境	3
2.4	L0 Cache 依赖检测	3
2.5	环境变量汇总	3
3	原生引擎构建	3
3.1	构建步骤	4
3.2	完整重建与产物一致性	4
3.3	鲲鹏平台默认启用的编译选项	4
3.4	可选的 C++ 单元测试	5
3.5	CMake 选项汇总	5
4	Java 主工程构建	5
4.1	编译与单元测试	5
4.2	打包 JAR	6
5	Benchmark 构建	6
5.1	WordCount Benchmark	6
5.2	NexMark Benchmark	6
5.3	(可选) JMH Benchmark	6
5.4	Benchmark 脚本依赖	7
6	一键构建与离线打包	7
7	常见问题与排查	7
8	附录	8
8.1	项目目录结构	8

1 文档概述

本文档给出 ForL0 State Backend 从基础环境准备、原生引擎编译、Java 主工程打包到基线 Benchmark 构建的完整流程，仅覆盖编译构建相关内容。集群部署、作业配置、运行期验证等使用层面内容由《使用说明书》单独说明；架构与数据结构设计由《设计说明书》给出。

1.1 目标平台

文中提到的所有路径、依赖包和命令均假定在如下环境下执行：

项目	要求
处理器	鲲鹏系列处理器
操作系统	openEuler
架构	aarch64 (ARM64)
内核特性	支持鲲鹏 L0 Cache 的内核驱动及 /dev/hisi_l0 或 /dev/l0 设备节点 (若计划启用 L0 HotCache)

L0 Cache 相关的硬件与驱动并非编译期依赖。若目标机器缺少 L0 硬件或内存池库，L0 HotCache 保持关闭，权威状态仍由原生 SwissTable 正常提供。

1.2 构建产物

按照本文档的构建流程，最终会得到下列产物：

产物	说明
libforl0_engine.so	原生状态引擎共享库 (aarch64 版本)
flink-statebackend-forL0-1.0-SNAPSHOT.jar	Java 控制层与 JNI 桥接打包的状态后端 JAR
wordcount-benchmark-*.jar	WordCount 基线性能测试 JAR
nexmark-flink-0.3-SNAPSHOT.jar	NexMark 基线性能测试 JAR

2 基础环境准备

基础环境准备包括系统构建工具、JDK、Maven、CMake、Flink 运行环境，以及 L0 Cache 相关的可选依赖。若构建机与部署机分离，基础构建工具可只安装在构建机上；JDK、Flink 与 L0 库一般需要部署到运行机。

2.1 系统构建工具

C++ 原生引擎采用 CMake + Make 构建，对 GCC 版本有一定要求。在 openEuler 上可通过包管理器直接安装：

```
sudo yum install -y gcc gcc-c++ cmake3 make

# 若默认仅有 cmake3 可执行文件, 建议创建 cmake 软链接
sudo ln -sf /usr/bin/cmake3 /usr/bin/cmake

gcc --version    # 要求 >= 7.0, 支持 C++17
```

部分发行版自带的 CMake 版本低于 3.14, 无法满足原生引擎构建。此时可使用官方 aarch64 预编译包:

```
wget https://github.com/Kitware/CMake/releases/download/v3.28.0/cmake-3.28.0-linux-aarch64.tar.gz
tar xzf cmake-3.28.0-linux-aarch64.tar.gz -C /opt/
export PATH=/opt/cmake-3.28.0-linux-aarch64/bin:$PATH
cmake --version
```

2.2 JDK 与 Maven

Java 主工程使用 JDK 1.8 与 Maven 3.6 及以上版本。在鲲鹏平台建议使用华为毕昇 JDK, 也可直接使用 OpenJDK 提供的 aarch64 包:

```
# OpenJDK (系统源)
sudo yum install -y java-1.8.0-openjdk-devel

# 或使用 BiSheng JDK (鲲鹏优化, 从华为镜像站下载 aarch64 版本后解压至 /opt)
export JAVA_HOME=/opt/bisheng-jdk1.8.0
export PATH=$JAVA_HOME/bin:$PATH

java -version
javac -version
```

Maven 使用官方二进制包安装:

```
wget https://archive.apache.org/dist/maven/maven-3/3.8.8/binaries/apache-maven-3.8.8-bin.tar.gz
tar xzf apache-maven-3.8.8-bin.tar.gz -C /opt/
export MAVEN_HOME=/opt/apache-maven-3.8.8
export PATH=$MAVEN_HOME/bin:$PATH
mvn --version
```

2.3 Apache Flink 运行环境

部署机需要具备 Apache Flink 1.20.3 二进制发行版。Flink 自身为 Scala 2.12 构建，无架构差异，可直接从官方下载：

```
wget https://archive.apache.org/dist/flink/flink-1.20.3/flink-1.20.3-bin-scala_2.12.tgz
tar xzf flink-1.20.3-bin-scala_2.12.tgz -C /opt/
export FLINK_HOME=/opt/flink-1.20.3
$FLINK_HOME/bin/flink --version
```

2.4 L0 Cache 依赖检测

鲲鹏 L0 Cache 加速依赖 L0 设备节点和内存池共享库 libl0mempool.so。二者均为运行期依赖，构建期无需参与链接，但在部署前应提前检查。

```
# 检查设备节点、共享库与可调用符号
bash docker/lib/l0_detector.sh

# 也可分别检查设备节点
ls -la /dev/hisi_l0 /dev/l0 2>/dev/null

# 检查共享库解析结果
ldconfig -p 2>/dev/null | grep libl0mempool || true
```

设备节点、共享库、必要符号及首次 L0 分配均通过后，运行时才会激活 HotCache。任一条件不满足时，非 strict 配置保持 HotCache 关闭，SwissTable 路径不受影响。构建期无需任何 L0 依赖参与。

2.5 环境变量汇总

为便于后续命令复用，建议将关键变量固化到 ~/.bashrc：

```
export JAVA_HOME=/opt/bisheng-jdk1.8.0
export MAVEN_HOME=/opt/apache-maven-3.8.8
export FLINK_HOME=/opt/flink-1.20.3
export PATH=$JAVA_HOME/bin:$MAVEN_HOME/bin:$FLINK_HOME/bin:$PATH
```

3 原生引擎构建

原生引擎是 ForL0 State Backend 的核心组件，包含 SwissTable 哈希存储、状态引擎、Copy-On-Write 快照实现以及 JNI 桥接层。原生引擎须先于 Java 主工程构建，Java 侧的测试与运行均依赖其产物。

3.1 构建步骤

原生引擎位于 `src/main/native` 下。交付构建与离线脚本使用仓库内 `Makefile`，推荐流程如下：

```
cd src/main/native
make clean
make -j$(nproc)
make install
```

`make install` 会将生成的 `libforl0_engine.so` 复制到 `src/main/resources/native/`，随后 Java 测试和打包流程即可引用该产物。`CMake` 主要用于构建原生单元测试，也会在共享库构建完成后复制同一产物。

3.2 完整重建与产物一致性

`native` 引擎的大量实现位于模板头文件中。头文件、JNI 声明或类型布局发生变化后，应执行 `make clean` 再重新构建，避免残留 `object` 与当前源码不一致。

部署前必须保持以下副本一致：`src/main/resources/native/libforl0_engine.so`、`docker/deploy/libforl0_engine.so`、`backend` JAR 内嵌 `native resource`，以及 `Flink lib//native/` 中的运行副本。仅替换 JAR 或仅替换外部 `.so` 都可能产生 Java/JNI/native 版本错配。

3.3 鲲鹏平台默认启用的编译选项

在 `aarch64` 上，`Makefile` 与 `CMake` 脚本会根据平台自动启用下列优化，不需要在命令行显式指定：

- `-O3 -DNDEBUG -march=native`：以鲲鹏 CPU 为目标的最高级别优化。
- `-DFORL0_NEON=1`：启用 NEON 路径的 SWAR 并行匹配实现，用于 `SwissTable` 的控制字节比较。
- `${CMAKE_DL_LIBS}` 链接：仅链接 `libdl`，以便运行时通过 `dlopen` 动态加载 `L0` 内存池库。

构建产物应为 `aarch64` 架构的 `ELF` 共享库，可通过如下命令核对：

```
file src/main/resources/native/libforl0_engine.so
# 预期：ELF 64-bit LSB shared object, ARM aarch64, ...
nm -D src/main/resources/native/libforl0_engine.so | grep Java_org_apache
| head
```

3.4 可选的 C++ 单元测试

原生单元测试由 CMake 构建，FORL0_BUILD_TESTS 默认启用：

```
cd src/main/native
rm -rf build && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -DFORL0_BUILD_TESTS=ON
make -j$(nproc)
ctest --output-on-failure
```

3.5 CMake 选项汇总

选项	默认值	说明
CMAKE_BUILD_TYPE	未指定	未指定时使用调试编译参数；生产交付必须显式设置为 Release。
FORL0_BUILD_TESTS	ON	置为 OFF 时跳过 C++ 单元测试可执行文件。

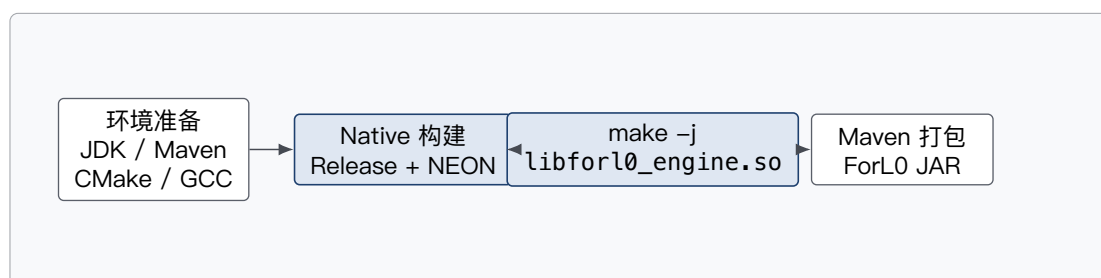


图 1 ForL0 State Backend 构建流水线

4 Java 主工程构建

原生引擎产物就绪后即可构建 Java 主工程。主工程负责 Flink 侧后端适配、JNI 调用入口以及测试用例打包。

4.1 编译与单元测试

基础编译与测试命令如下：

```
# 回到项目根目录
cd $PROJECT_ROOT

# 仅编译源码
mvn clean compile

# 编译 + 运行单元测试
```

```
mvn test
```

Maven Surefire 已预置 `-Djava.library.path=src/main/resources/native`，测试阶段会自动加载原生库，无需额外配置。

4.2 打包 JAR

构建最终交付 JAR：

```
mvn package -DskipTests
ls target/flink-statebackend-forL0-1.0-SNAPSHOT.jar
```

生产交付建议使用带测试的完整构建：

```
mvn clean package
```

若构建机无法访问公共 Maven 仓库，需在 `~/.m2/settings.xml` 中配置内网镜像，或先行准备离线仓库。

5 Benchmark 构建

ForL0 State Backend 交付物中包含两套基线性能测试：WordCount 基线和 NexMark 基线。它们与主工程解耦，可独立构建，也可在部署后单独提交至 Flink 集群。

5.1 WordCount Benchmark

```
cd benchmark/wordcount
mvn clean package -DskipTests
ls target/wordcount-benchmark-*.jar
```

5.2 NexMark Benchmark

```
cd benchmark/nexmark-src
mvn clean package -DskipTests
ls nexmark-flink/target/nexmark-flink-0.3-SNAPSHOT.jar
ls nexmark-flink/target/nexmark-flink-bin/nexmark-flink/bin
```

5.3 (可选) JMH Benchmark

Flink 官方的 JMH Benchmark 用于回归对比，可按需构建：

```
cd benchmark/flink-benchmarks
./mvnw clean package -DskipTests
```


5.4 Benchmark 脚本依赖

Benchmark 运行脚本使用 Python 编写。运行环境需安装 Python 3.8 及以上版本：

```
sudo yum install -y python3 python3-pip
cd benchmark/
pip3 install -r requirements.txt

# 封闭网络环境下使用随源码交付的离线安装包
pip3 install --no-index --find-links=offline-packages/ -r requirements.
txt
```

6 一键构建与离线打包

仓库提供正式的一键构建与离线打包入口。该脚本依次构建后端 JAR、native 库、Benchmark 产物，并收集离线运行依赖：

```
cd $PROJECT_ROOT
chmod +x docker/package_offline_bundle.sh

# 输出到指定目录，并固定目标 Python ABI
./docker/package_offline_bundle.sh \
  --arch arm64 \
  --python-version 3.10 \
  --output-dir "$PWD/offline-artifacts/forl0-offline-linux-arm64-py310"
```

7 常见问题与排查

现象	可能原因	处理方式
CMake Error: Could not find JNI unrecognized option '-march=native'	JAVA_HOME 未正确设置 GCC 版本低于 7.0	export JAVA_HOME=/opt/bisheng-jdk1.8.0 后重新配置 升级 GCC (sudo yum install devtoolset-8) 或使用 BiSheng 编译器
jni.h: No such file or directory cannot find -lstdc++fs	缺少 JDK 开发包 libstdc++ 版本过低	安装 java-1.8.0-openjdk-devel 或确认 BiSheng JDK 路径完整 安装 libstdc++-devel 或切换至新版 GCC
mvn test 抛出 UnsatisfiedLinkError NexMark 编译卡在依赖下载	原生库缺失或与 Java 代码版本不一致 仓库不可达	重新执行 § 3 节的 make clean、make 和 make install 配置内网 Maven 镜像或使用离线仓库

8 附录

8.1 项目目录结构

以下目录结构摘录了本文档涉及的构建相关路径，其他内容参见项目源码：

```
forL0-state-backend/
|-- pom.xml                      # Java 主工程 POM
|-- src/
|   |-- main/
|   |   |-- java/                # Java 源码 (Flink 适配与 JNI 入口)
|   |   |   |-- org/apache/flink/state/forl0/
|   |   |-- native/              # 原生引擎源码
|   |   |   |-- CMakeLists.txt
|   |   |   |-- Makefile
|   |   |   |-- engine/          # 引擎与 HotCache 实现
|   |   |   |   |-- swiss_table.h # Swiss Tables 哈希表
|   |   |   |   |-- allocator.h  # 分配器抽象
|   |   |   |   |-- hot_cache.h  # L0 HotCache
|   |   |   |   |-- state_engine.h
|   |   |-- jni/                 # JNI 桥接
|   |   |-- checkpoint/          # 快照序列化
|   |   |   |-- test/            # C++ 单元测试
|   |   |   |-- resources/native/ # native 构建复制的共享库
|   |   |-- test/java/           # Java 测试代码
|-- benchmark/
|   |-- wordcount/               # WordCount Benchmark
|   |-- nexmark-src/             # NexMark Benchmark
|   |-- flink-benchmarks/        # JMH Benchmark
|   |-- scripts/                 # Python 运行脚本
|-- target/                      # Maven 产物
```